Project Overview

Project title: HR Employee Attrition Prediction.

Notebook used: finalcodemlproject.ipynb.

Dataset used: Imperfect_HR_Attrition (1) (1).csv.

Raw dataset shape: 1749 rows and 35 columns.

Goal: predict whether an employee belongs to the Attrition Yes or No class using employee, job, compensation, satisfaction, and tenure-related fields.

Target column: Attrition. The model treats this as a binary classification problem with classes No and Yes.

Target distribution in the raw CSV: Attrition No 1266 Yes 483

The project now has two user interfaces. app.py is the Streamlit Python app that reuses the notebook pipeline. index.html is a one-page browser dashboard with EDA charts and a JavaScript prediction form based on the trained Logistic Regression parameters.

Notebook Code Flow

Cell 1 imports basic Kaggle environment helpers. It is not needed for local deployment, so notebook_bridge.py skips it when running the notebook for the apps.

The import cell loads pandas and numpy for data handling, matplotlib and seaborn for plots, scikit-learn utilities for splitting/scaling/models/metrics, and imblearn.SMOTE for class balancing.

The loading cell reads the HR attrition CSV into df and prints the first rows, dataset shape, info, null counts, and descriptive statistics. This gives the first understanding of row count, column types, and missing data.

Duplicate handling: df.duplicated().sum() is checked, then df.drop_duplicates(inplace=True) removes duplicate rows. Raw duplicate count found: 97.

Missing value handling: numeric columns are imputed with mean and categorical columns with most frequent value. Missing values found in raw data: DistanceFromHome: 109, EnvironmentSatisfaction: 135, Gender: 69, JobSatisfaction: 141, MonthlyIncome: 21, NumCompaniesWorked: 157, PercentSalaryHike: 89, RelationshipSatisfaction: 142, TotalWorkingYears: 1, TrainingTimesLastYear: 127, WorkLifeBalance: 130, YearsWithCurrManager: 83

Label encoding: object columns are converted into numeric values using LabelEncoder. Encoded columns include: Attrition, BusinessTravel, Department, EducationField, Gender, JobRole, MaritalStatus, Over18, OverTime. This is required because the selected machine learning algorithms expect numeric input.

Constant and ID column detection is done by checking the number of unique values in each column. A column with one unique value is constant. A column with unique values equal to total rows is ID-like.

Dropped columns: EmployeeCount, EmployeeNumber, Over18, and StandardHours. EmployeeCount, Over18, and StandardHours do not vary meaningfully, while EmployeeNumber is an identifier and should not guide prediction.

EDA plots in the notebook include a correlation heatmap, decision tree visualization, Random Forest feature importance, model metric comparison, and before/after SMOTE comparison.

Feature Engineering

Feature selection is simple and transparent: X is created by dropping Attrition from df, and y is created from df['Attrition'].

Final model feature count after preprocessing: 30. Features used: Age, BusinessTravel, DailyRate, Department, DistanceFromHome, Education, EducationField, EnvironmentSatisfaction, Gender, HourlyRate, JobInvolvement, JobLevel, JobRole, JobSatisfaction, MaritalStatus, MonthlyIncome, MonthlyRate, NumCompaniesWorked, OverTime, PercentSalaryHike, PerformanceRating, RelationshipSatisfaction, StockOptionLevel, TotalWorkingYears, TrainingTimesLastYear, WorkLifeBalance, YearsAtCompany, YearsInCurrentRole, YearsSinceLastPromotion, YearsWithCurrManager

train_test_split(X, y, test_size=0.2, random_state=42) creates an 80 percent training set and 20 percent test set. random_state=42 makes the split reproducible.

StandardScaler is fitted only on X_train and then applied to X_train and X_test. This avoids data leakage because the test set is not used to learn the scaling parameters.

Scaling is especially important for Logistic Regression and KNN because these models are affected by feature magnitude. It also keeps model comparisons consistent.

Models and Evaluation

Logistic Regression is used as a linear baseline model. It learns weighted relationships between scaled employee features and the probability of attrition.

Random Forest is used because it can capture non-linear relationships and interactions between features. It also provides feature importance values used in the dashboard.

CART Decision Tree is used as an interpretable tree model with criterion='gini' and max_depth=5. The max depth limits overfitting and keeps the tree readable.

KNN is used as a distance-based classifier with n_neighbors=5. Since KNN depends on distances, scaling is necessary before training.

XGBoost code exists in the notebook. The local environment used here does not have xgboost installed, so notebook_bridge.py skips only the XGBoost cells and keeps the rest of the workflow working.

The evaluate_model function calculates Accuracy, Precision, Recall, F1 Score, ROC AUC, classification report, and confusion matrix. These metrics are more informative than accuracy alone because attrition classes are imbalanced.

Accuracy measures total correct predictions. Precision measures how many predicted attrition cases were actually attrition. Recall measures how many actual attrition cases were found. F1 balances precision and recall. ROC AUC summarizes class separation.

SMOTE is applied only to training data after scaling. This creates synthetic minority-class samples in the training set while keeping the test set untouched for fair evaluation.

Before SMOTE results:

Model Accuracy Precision Recall F1 Score ROC AUC Logistic Regression 0.8097 0.7297 0.5567 0.6316 0.7356 Random Forest 0.9215 0.9383 0.7835 0.8539 0.8811 CART Decision Tree 0.7946 0.7042 0.5155 0.5952 0.7129 KNN 0.7764 0.6825 0.4433 0.5375 0.6789

After SMOTE results:

Model Accuracy Precision Recall F1 Score ROC AUC LR + SMOTE 0.7583 0.5620 0.7938 0.6581 0.7687 RF + SMOTE 0.9275 0.9294 0.8144 0.8681 0.8944 DT + SMOTE 0.7553 0.5645 0.7216 0.6335 0.7454 KNN + SMOTE 0.7372 0.5316 0.8660 0.6588 0.7749

Models and Evaluation continued

Streamlit App Code

app.py is the Python Streamlit frontend. It does not rewrite the complete ML code. Instead, it calls load_notebook_namespace() from notebook_bridge.py.

st.set_page_config sets the browser title, icon, and layout. st.cache_resource caches notebook execution so the notebook does not retrain on every small UI refresh.

The app reads raw_df, X, scaler, label_encoders, and trained models from the notebook namespace. This keeps the app connected to the original notebook logic.

The model dictionary exposes Random Forest, Logistic Regression, CART Decision Tree, KNN, and their SMOTE versions when available.

For each feature in X.columns, the app creates a Streamlit input widget. Categorical columns become selectbox widgets using the original LabelEncoder classes. Numeric columns become number_input widgets using actual min, max, and median values from the CSV.

When the user clicks Predict Attrition, the app builds a one-row DataFrame in the same feature order as training. Categorical values are transformed through the same label encoders, then scaler.transform() applies the training scaler.

The selected trained model predicts the class. If predict_proba exists, the app also displays class probabilities as a bar chart.

Notebook Bridge Code

notebook_bridge.py is the connector between the existing ipynb and the new apps.

BASE_DIR, NOTEBOOK_PATH, and DATA_PATH locate the notebook and CSV relative to the project folder, so the project can run from this directory without using the old Kaggle or D:/ML Project path.

_prepare_source() modifies notebook cells at runtime. It replaces the old CSV path with the local CSV path, skips Kaggle input listing, skips heavy notebook-only plot cells, and skips XGBoost cells when xgboost is unavailable.

load_notebook_namespace() reads the notebook with nbformat, executes each usable code cell, suppresses print/display output, closes matplotlib figures, and returns the namespace dictionary containing trained variables.

This approach satisfies the requirement to connect the ipynb instead of rewriting the full Streamlit ML pipeline.

HTML Frontend Code

index.html is a one-page dashboard that can open directly in a browser. It does not require Streamlit, Flask, internet, or external JavaScript libraries.

The top part contains navigation links for Predict, Overview, EDA Graphs, Models, Features, and Tables.

The EDA chart section uses Canvas. Buttons switch between Attrition distribution, Department distribution, Job Role distribution, and Missing Values.

The relationship chart section shows OverTime vs Attrition, Age Group vs Attrition, and Average Monthly Income by Attrition using data computed from the actual CSV.

The model section draws before-SMOTE and after-SMOTE F1 score charts from the notebook result tables.

The feature section draws top Random Forest feature importance from rf.feature_importances_ and X.columns.

The prediction form in HTML uses the trained Logistic Regression model. generate_frontend_and_docs.py exports the model coefficients, intercept, scaler means, scaler scales, feature metadata, and target classes into JavaScript.

When Predict Attrition is clicked, JavaScript collects values, label-encodes categorical selections by their class index, scales each feature using (value - scalerMean) / scalerScale, calculates the Logistic Regression score, applies sigmoid, and returns No or Yes based on a 0.5 threshold.

Generator Script Code

generate_frontend_and_docs.py is the reproducibility script. Running it regenerates index.html and HR_Attrition_Code_Documentation.pdf from the current notebook and CSV.

build_html() reads dataset summaries, missing values, model metrics, feature importance, chart payloads, and Logistic Regression parameters, then writes a complete self-contained HTML file.

build_pdf() writes this documentation using matplotlib.backends.backend_pdf.PdfPages. This avoids requiring extra PDF libraries.

add_pdf_page() wraps long text onto PDF pages and automatically continues on a new page when needed.

If the dataset or notebook changes later, running python generate_frontend_and_docs.py updates both the dashboard and the documentation.

Project Files

finalcodemlproject.ipynb: original ML notebook containing data loading, cleaning, preprocessing, training, evaluation, SMOTE, and visualizations.

Imperfect_HR_Attrition (1) (1).csv: source HR attrition dataset.

notebook_bridge.py: executes the notebook safely for local app use and returns trained variables.

app.py: Streamlit app for interactive prediction using notebook-trained models.

index.html: standalone interactive dashboard with EDA graphs and browser prediction.

generate_frontend_and_docs.py: regenerates the HTML frontend and documentation PDF.

HR_Attrition_Code_Documentation.pdf: detailed code documentation and team division.

Three Member Work Division

Member 1: Data Understanding and EDA. Responsibilities: loading the CSV in the notebook, checking df.head(), df.shape, df.info(), null values, duplicate rows, and df.describe(). This member also prepared EDA interpretation using target distribution, department distribution, job role distribution, overtime relation, age group relation, income comparison, correlation heatmap, and missing-value analysis.

Member 1 output: dataset understanding section, EDA graph section in HTML, and explanation of what the data contains before modelling.

Member 2: Preprocessing and Model Building. Responsibilities: dropping duplicate rows, imputing numeric columns with mean, imputing categorical columns with most frequent value, label encoding object columns, identifying constant/ID-like columns, dropping EmployeeCount, EmployeeNumber, Over18, and StandardHours, splitting X and y, applying StandardScaler, and training Logistic Regression, Random Forest, CART Decision Tree, KNN, and optional XGBoost.

Member 2 output: cleaned modelling dataset, trained classifiers, saved notebook variables, and Random Forest feature importance.

Member 3: Evaluation, UI, Deployment, and Documentation. Responsibilities: writing evaluation functions, generating confusion matrices, classification reports, before/after SMOTE metric tables, SMOTE balancing, Streamlit app, notebook bridge, interactive HTML dashboard, JavaScript prediction logic, and this PDF documentation.

Member 3 output: app.py, notebook_bridge.py, index.html, generate_frontend_and_docs.py, and HR_Attrition_Code_Documentation.pdf.

This division is written according to the completed project work and can be used in presentation or viva to explain individual contributions clearly.